Quantum-Assisted Resource Allocation for Multi-Agent AI Systems

Kien Nguyen
University of Waterloo

November 3, 2026

Abstract

As multi-agent AI systems grow in complexity, efficiently allocating tasks across heterogeneous agents under token budget constraints becomes a critical challenge. We formulate this problem as a variant of the **Generalized Assignment Problem (GAP)** and investigate whether quantum optimization can provide effective solutions. Applying the **Variational Quantum Generalized Assignment Problem (VQGAP)** and its logarithmically encoded extension (**VQGAPe**) of Mastroianni et al. (2025), we show how the standard QUBO reformulation's qubit overhead—caused by binary-expanding integer slack variables—can be eliminated by moving capacity constraint evaluation onto the classical side of the hybrid loop, while compact binary encoding shrinks the register from NM down to $N\lceil \log_2 M \rceil$ qubits. The resulting approach dramatically lowers circuit requirements, making it feasible on near-term (NISQ) hardware. We describe the full formulation, penalty structure, and the decoding procedure for retrieving optimal assignments from quantum measurements.

1 Introduction

1.1 Research Question

Can quantum optimization algorithms efficiently solve resource-constrained task allocation problems in multi-agent AI systems?

1.2 Problem Setting

Modern AI pipelines increasingly rely on multi-agent systems, where specialized agents—each with distinct capabilities and computational costs—collaborate to handle complex workflows. A key operational challenge is *how to assign tasks to agents* so as to maximize overall completion quality while staying within each agent's token budget.

We model this as a variant of the **Generalized Assignment Problem (GAP)** and solve it using the **Variational Quantum Generalized Assignment Problem (VQGAP)** framework of Mastroianni et al. [1].

1.3 Mathematical Formulation

Consider $N \in \mathbb{Z}_+$ tasks and $M \in \mathbb{Z}_+$ agents.

We will use the following notations for ease of notation:

- $[N] = \{1, \ldots, N\}$
- $[M] = \{1, \ldots, M\}$

Let $x_{nm} \in \{0, 1\}$ indicate whether task $n \in [N]$ is assigned to agent $m \in [M]$. Each agent m has a token budget b_m; assigning task n to agent m consumes a_{nm} tokens and yields an expected success rate $c_{nm} \in [0, 1]$.

Since $c_{nm} \in [0, 1]$ while token costs a_{nm} can be orders of magnitude larger, we define a normalized cost

$$\bar{a}_{nm} = \frac{a_{nm}}{a_{\max}}$$

where

$$a_{\max} = \max_{n \in [N],\, m \in [M]} a_{nm}$$

so that $\bar{a}_{nm} \in [0,1]$ and both objectives live on the same scale.

Our assignment must balance two competing objectives.

The first is to **maximize completion quality**:

$$\begin{aligned} \max_x \quad & \sum_{n\in[N],\, m\in[M]} c_{nm}\, x_{nm} \\ \text{s.t.} \quad & \sum_{m\in[M]} x_{nm} = 1, \quad \forall n \in [N] \\ & \sum_{n\in[N]} a_{nm}\, x_{nm} \le b_m, \quad \forall m \in [M] \\ & x_{nm} \in \{0,1\} \end{aligned} \tag{P_1}$$

The second is to **minimize total (normalized) token consumption**:

$$\begin{aligned} \min_x \quad & \sum_{n\in[N],\, m\in[M]} \bar{a}_{nm}\, x_{nm} \\ \text{s.t.} \quad & \sum_{m\in[M]} x_{nm} = 1, \quad \forall n \in [N] \\ & \sum_{n\in[N]} a_{nm}\, x_{nm} \le b_m, \quad \forall m \in [M] \\ & x_{nm} \in \{0,1\} \end{aligned} \tag{P_2}$$

Since both problems share the same feasible set, we can merge them into a single objective via a weighted sum with trade-off parameter $\lambda \in [0,1]$, which controls the emphasis on cost efficiency versus quality:

$$\begin{aligned} \min_x \quad & \sum_{n\in[N],\, m\in[M]} \left[\lambda\, \bar{a}_{nm} - (1-\lambda)\, c_{nm}\right] x_{nm} \\ \text{s.t.} \quad & \sum_{m\in[M]} x_{nm} = 1, \quad \forall n \in [N] \qquad \text{(each task assigned once)} \\ & \sum_{n\in[N]} a_{nm}\, x_{nm} \le b_m, \quad \forall m \in [M] \qquad \text{(token budget)} \\ & x_{nm} \in \{0,1\}, \quad \forall n \in [N], \forall m \in [M] \end{aligned} \tag{P_{GAP}}$$

1.4 QUBO Reformulation

Variational quantum algorithms require a **Quadratic Unconstrained Binary Optimization (QUBO)** formulation.

Defining the combined weight

$$w_{nm} = \lambda \bar{a}_{nm} - (1-\lambda) c_{nm}, \quad \forall n \in [N], \forall m \in [M]$$

the standard approach converts the constraints into quadratic penalty terms:

$$H(x,y) = \sum_{n\in[N],\ m\in[M]} w_{nm} x_{nm} + C \sum_{n\in[N]} \left(1 - \textstyle\sum_m x_{nm}\right)^2 + C \sum_{m\in[M]} \left(b_m - \textstyle\sum_n a_{nm} x_{nm} - s_m\right)^2$$

where

$$C = 1 + \sum_{n,m} |w_{nm}|$$

Further, in the above formulation, each slack variable $s_m, \forall m \in [M]$ represents the unused token budget of agent m.

Binary expansion of slack variables.

In a standard QUBO, the integer slack $s_m \in \{0, \ldots, b_m\}, \forall m \in [M]$ must itself be encoded in binary. Using $K_m = \lceil \log_2(b_m + 1) \rceil$ auxiliary binary variables y_{mk}, we write

$$s_m = \sum_{k=0}^{K_m - 1} v_{mk}\, y_{mk},$$

where

$$v_{mk} = \begin{cases} 2^k, & k < K_m - 1 \\ b_m + 1 - 2^{K_m - 1}, & k = K_m - 1 \end{cases}$$

is a bounded binary encoding that covers all values in $\{0, \ldots, b_m\}$ exactly. This adds $\sum_m K_m$ slack qubits on top of the NM assignment qubits—a serious bottleneck when token budgets b_m are large.

1.5 VQE vs. VQGAP: Key Differences

The central bottleneck in applying a standard Variational Quantum Eigensolver (VQE) to P_{GAP} is the qubit count. Every binary variable must occupy

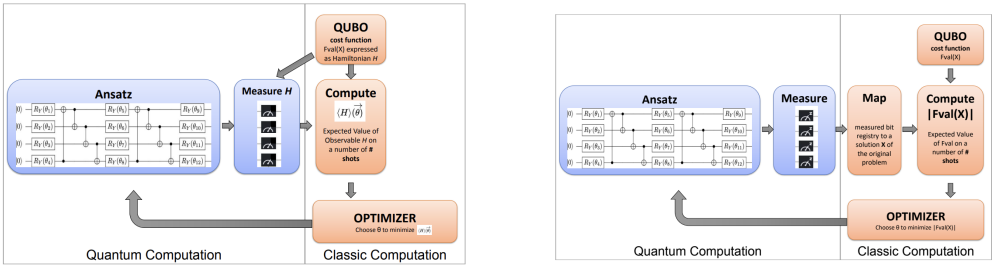
its own qubit, so the quantum register must hold all NM assignment variables *and* all slack bits:

$$Q_{\text{VQE}} = \underbrace{NM}_{\text{assignment}} + \underbrace{\sum_{m=1}^{M} \lceil \log_2(b_m + 1) \rceil}_{\text{slack (binary-expanded)}}$$

In practice, with token budgets in the thousands, the slack term alone can dwarf the assignment term, making the approach infeasible on near-term (NISQ) hardware.

VQGAP eliminates the slack qubits entirely by moving the capacity check off the quantum register and onto the classical computer. The quantum circuit only encodes the NM assignment variables; everything else is handled post-measurement:

$$Q_{\text{VQGAP}} = NM$$



(a) Standard VQE (b) VQGAP

Figure 1: Architecture comparison: standard VQE (left) encodes both assignment and slack variables as qubits, while VQGAP (right) restricts the quantum register to assignment bits only and decodes slack classically, eliminating the $\sum_m \lceil \log_2(b_m + 1) \rceil$ slack qubit overhead. Figures adapted from Mastroianni et al. [1].

1.6 VQGAP Formulation and Solution Retrieval

The VQGAP framework [1] avoids the slack qubit bottleneck by keeping s_m as a *classical* variable. Only the NM assignment bits x_{nm} are placed on the

quantum register.

A parameterized quantum circuit $U(\vec{\theta})$ is trained using a classical optimizer (e.g., COBYLA) to minimize the expected cost $\langle F_{\text{val}} \rangle$ over measurement shots. During both training evaluation and final solution extraction, the optimal assignment is retrieved as follows:

1. **Measurement:** Sample K bitstrings $\{x^{(k)}\}_{k=1}^{K}$ from the circuit $U(\vec{\theta}^*)$. Each bitstring has length NM and directly encodes the assignment matrix.

2. **Residual Slack Computation:** For each agent m and shot k, the remaining token budget is computed classically:

$$s_m^{(k)} = \max\left(0,\ b_m - \sum_{n=1}^{N} a_{nm}\, x_{nm}^{(k)}\right)$$

   This guarantees that feasible assignments incur zero capacity penalty, while infeasible ones trigger a quadratic violation penalty.

3. **Cost Evaluation and Selection:** Each sample is evaluated using the extended cost function:

$$\begin{aligned} F_{\text{val}}(x^{(k)}, s^{(k)}) = & \sum_{n\in[N],\, m\in[M]} w_{nm} x_{nm}^{(k)} \\ & + C \sum_{n\in[N]} \left(1 - \sum_{m\in[M]} x_{nm}^{(k)}\right)^2 \\ & + C \sum_{m\in[M]} \left(b_m - \sum_{n\in[N]} a_{nm} x_{nm}^{(k)} - s_m^{(k)}\right)^2 \end{aligned}$$

   with unified penalty coefficient:

$$C = 1 + \sum_{n\in[N],\, m\in[M]} |w_{nm}|$$

   The optimal assignment corresponds to the minimum-cost bitstring:

$$k^* = \underset{k\in\{1,\ldots,K\}}{\arg\min}\ F_{\text{val}}\big(x^{(k)}, s^{(k)}\big)$$

If $x^{(k^*)}$ satisfies all capacity constraints, it is returned as the final assignment. Otherwise, the best feasible bitstring among all K shots is selected.

1.7 Logarithmic Compression via VQGAPe

While standard VQGAP reduces the circuit register to NM qubits by moving the slack variable evaluation to classical post-processing, the number of required qubits still scales linearly with the number of agents M. In many practical multi-agent settings, this becomes prohibitive as the pool of candidate agents grows.

To address this, Mastroianni et al. [1] introduced the **VQGAPe** (encoded) framework. The key insight is that the one-to-one assignment constraint

$$\sum_{m \in [M]} x_{nm} = 1$$

requires exactly one agent to be chosen per task.

Therefore, rather than allocating a separate qubit for every potential task-agent pair (n, m), we can encode the index of the assigned agent using binary representation.

Specifically, for each task $n \in [N]$, we allocate

$$L = \lceil \log_2 M \rceil$$

qubits (or $\lceil \log_2(M+1) \rceil$ if an explicit unassigned state is permitted).

The total quantum register size is compressed to:

$$Q_{\text{VQGAPe}} = N \lceil \log_2 M \rceil$$

which scales *logarithmically* with the number of agents M, rather than linearly.

Decoding and Optimization Workflow:

1. **Measurement:** Measuring the NL qubits yields bit values $e_{nl}^{(k)} \in \{0, 1\}$ for each task $n \in [N]$ and qubit index $l \in \{1, \ldots, L\}$.

2. **Index Decoding:** The assigned agent index $m_n^{(k)} \in [M]$ for task n is decoded directly from binary:

$$m_n^{(k)} = 1 + \sum_{l=1}^{L} 2^{l-1} e_{nl}^{(k)}$$

The corresponding binary decision variables are mapped as:

$$x_{nm}^{(k)} = \begin{cases} 1, & \text{if } m = m_n^{(k)} \\ 0, & \text{otherwise} \end{cases}$$

Because every measurement directly decodes into exactly one agent per task, the task assignment constraint $\sum_{m \in [M]} x_{nm} = 1$ is automatically satisfied by construction, substantially restricting the quantum search space to valid assignment subspaces.

3. **Residual Capacity and Evaluation:** As in standard VQGAP, the remaining token budgets $s_m^{(k)}$ and the extended objective $F_{\text{val}}(x^{(k)}, s^{(k)})$ are evaluated classically.

This logarithmic compression dramatically reduces circuit width, making variational optimization on NISQ hardware viable for larger agent pools.

References

[1] C. Mastroianni, F. Plastina, J. Settino, and A. Vinci, "Efficient Variational Quantum Algorithms for the Generalized Assignment Problem," *arXiv preprint arXiv:2511.02739*, 2025.